

Relatório do Analisador Léxico (Etapa 1) – GCC130

Giovane Felipe Godoi Oliveira & Luiz Fellipe Resende Lima
Universidade Federal de Lavras (UFLA)
Departamento de Ciência da Computação
Lavras, MG, Brasil

Resumo

Este relatório apresenta o projeto e a implementação de um analisador léxico construído utilizando a ferramenta Flex para uma mini linguagem imperativa. São discutidas as decisões de projeto, especificação da sintaxe, tratamento de erros, dificuldades encontradas, além de diagramas de transição e um teste completo de execução.

1 Especificação da Linguagem Proposta

A mini linguagem imperativa suporta declaração de variáveis, estruturas de controle e comandos de E/S.

- **Tipos de Dados:** int e float.
- **Palavras-chave:** if, else, while, print, read, return.
- **Operadores:** Aritméticos (+, -, *, /, %, =), Relacionais (>=, <=, !=, <>, ==, <, >) e Lógicos (&&, ||, !).
- **Pontuação e Delimitadores:** ;, ,, (,), {, }.
- **Identificadores e Constantes:** Identificadores iniciam por letra seguida de letras, dígitos ou underscore. Suporta inteiros e decimais com notação científica opcional (e.g., 12.3E+2).

Exemplo de sintaxe:

```
int x; float y;  
x = 10;  
if (x >= 5) { print(x); }
```

2 Decisões de Projeto

- **Tabela de Símbolos:** Implementada via vetor de estruturas (Símbolo), armazenando lexema, linha e coluna. Apenas identificadores são inseridos para evitar redundância com palavras reservadas.
- **Controle de Linha e Coluna:** Utilizou-se a variável yylineno do Flex para contagem de linhas e a variável global column_number incrementada pelo tamanho do lexema.
- **Comentários e Espaços:** Espaços e comentários de linha única (//) ou multilinha (/* ... */) são desconsiderados. O processamento de novas linhas dentro de comentários multilinhas é gerenciado pela sub-rotina atualizaColuna().
- **Valores Negativos:** O sinal negativo é tratado pelo operador aritmético de subtração (MINUS), delegando o agrupamento unário para a etapa sintática/semântica.
- **Tratamento de Erros:** Caracteres inválidos são capturados pela regra genérica, gerando mensagem formatada com linha e coluna. Foi implementado também uma regra para o tratamento com especificidade quanto ao id mal formado: não é permitido criar um id iniciando com número.
- **Notação científica:** Números com expoentes "E" ou "e" são considerados como floats, isso para ter uma maior representação de números de forma simplificada.
- **Representação de IDS:** Os IDS devem ser iniciados com uma letra e podem possuir diversas letras, números e underscores - mas nunca iniciarem com número ou underscore.

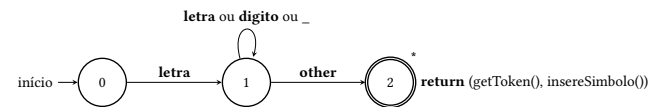
3 Dificuldades Encontradas

A principal dificuldade consistiu no rastreamento exato da coluna em comentários de múltiplas linhas contendo quebras de linha (\n).

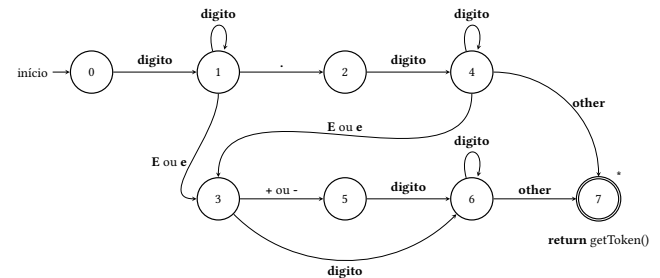
A solução consistiu na varredura caractere a caractere em atualizaColuna() para reiniciar a coluna sempre que uma nova linha fosse encontrada.

4 Diagramas de Transição

4.1 Identificadores e Palavras-Chave (id)



4.2 Constante Ponto Flutuante com Expoente (float)



5 Arquivo de Teste e Saída

Entrada (teste.txt):

```
/* Teste Lexico */  
int a = 10;  
float b = 2.5E+1;  
while( a > 0){  
    print(b);  
}  
if(a == b);  
return 0;
```

Saída Gerada:

```
2(1): int (TYPE)  
2(5): a (ID)  
2(7): = (ASSIGN)  
2(9): 10 (NUM_INT)  
2(11): ; (SEMI)  
3(1): float (TYPE)  
3(7): b (ID)  
3(9): = (ASSIGN)  
3(11): 2.5E+1 (NUM_FLOAT)  
3(17): ; (SEMI)  
4(1): while (WHILE)  
4(6): ( LPAREN)  
4(8): a (ID)  
4(10): > (RELOP)  
4(12): 0 (NUM_INT)  
4(13): ) (RPAREN)  
4(14): { (LBRACE)  
5(5): print (PRINT)  
5(10): ( LPAREN)  
5(11): b (ID)  
5(12): ) (RPAREN)  
5(13): ; (SEMI)  
6(1): } (RBRACE)  
8(1): if (IF)  
8(3): ( LPAREN)  
8(4): a (ID)  
8(6): == (RELOP)  
8(9): b (ID)  
8(10): ) (RPAREN)  
8(11): ; (SEMI)  
9(1): return (RETURN)  
9(8): 0 (NUM_INT)  
9(9): ; (SEMI)
```

```
=== TABELA DE SÍMBOLOS LEXICA ===  
ID LEXEMA LINHA COLUNA  
0 a 2 5  
1 b 3 7
```